AI-POWERED FILE ORGANIZER APPLICATION

MR. SARUN KHUMTHAI
MR. SUPAKORN TUNGPATOMPRAMOTE
MR. PUWADECH INTONG

A PROJECT SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR
THE DEGREE OF BACHELOR OF ENGINEERING (COMPUTER ENGINEERING)
FACULTY OF ENGINEERING
KING MONGKUT'S UNIVERSITY OF TECHNOLOGY THONBURI
2025

AI-powered file organizer application

Mr. Sarun Khumthai
Mr. Supakorn Tungpatompramote
Mr. Puwadech Intong

A Project Submitted in Partial Fulfillment
of the Requirements for
the Degree of Bachelor of Engineering (Computer Engineering)
Faculty of Engineering
King Mongkut's University of Technology Thonburi
2025

Project Committee

..... Project Advisor
(Suthathip Maneewongvatana, Ph.D.)

..... Committee Member
(Name, Title)

..... Committee Member
(Name, Title)

..... Committee Member
(Name, Title)

Project Title	AI-powered file organizer application
Credits	3
Member(s)	Mr. Sarun Khumthai
	Mr. Supakorn Tungpatompramote
	Mr. Puwadech Intong
Project Advisor	Suthathip Maneewongvatana, Ph.D.
Program	Bachelor of Engineering
Field of Study	Computer Engineering
Department	Computer Engineering
Faculty	Engineering
Academic Year	2025

Abstract

The project, “AI-powered file organizer application”, is motivated by the fact that nowadays almost everyone owns a computer and inevitable to create or handle “files.” The number of them is enormous, managing them can become troublesome. This project is going to address this common problem by developing a desktop application that can automatically read and understand file’s content to organize files with more precision. Our objective then is for this solution to assist with and reduce the burden of manual file management. Furthermore, aside from organization, it may also integrate AI to perform more advanced actions which aims to establish a foundation for agentic systems related to file management such as summarization, note taking, searching or calendar taking. Everything must be done locally, bringing greater convenience and ease to users. The performance evaluation of the “AI-powered file organizer application” showed that the overall performance was at the level of effectively solving the problem with minor improvements possible (mean score of 4.31) and standard deviation was 0.19, indicating that the experts’ opinions regarding the application’s performance were similar and consistent. The experts also provided overall comments on the “AI-powered file organizer application”, stating that the application has academic value and can be applied in real-world use. It can be further developed as an Open-Source Project. The application has a very strong foundation and is ready for personal use or small teams.

Keywords: AI-powered file organizer application / File organization / Classification / LLM / RAG

ACKNOWLEDGMENTS

This project would not have been possible with the support of many people. Firstly, we would like to express our sincere gratitude to our advisor, Suthathip Maneewongvatana, Ph.D., for her guidance, time, and support throughout this project. We would also like to thank every committee member who gives us valuable insights and feedback alongside our advisor throughout the entire semester, which is crucial to the development and success of this project.

A special thanks goes to three experts, Saenguthai Mortho, Asst. Ph.D., Mr. Sonthaya Kueakhiri and Mr. Theerawat NaNakhon, who give their time and expertise to us for the Expert Evaluation form and its feedback.

Lastly, we would like to thank everyone involved in the making of this project including those who cheer us on for their constant support and encouragement. Thank you from the bottom of our heart.

CONTENTS

PAGE

LIST OF TABLES

TABLE **PAGE**

LIST OF FIGURES

FIGURE **PAGE**

CHAPTER 1 INTRODUCTION

1.1 Motivation

At present, as the world enters the information age, technological knowledge has become widespread to all. Almost everyone owns at least one computing device. As a result, the creation and storage of digital data have increased continuously and on a massive scale. Users may have to deal with various types of files, such as documents (pdf, .docx, .txt), images (.png, .jpg), presentations (.pptx), and spreadsheets (.xlsx), which are ones that are widely used and exchanged in daily life. Each user has a different way of organizing files, but a common problem is that they have to move files manually after downloading them. If there are only a few files, this may not be a major issue. However, when the number of files increases, manual file management becomes a repetitive and time-consuming process prone to errors. Leaving files accumulated in the Downloads folder is not an appropriate method. Therefore, we aim to develop an automated file management system that can read and understand file contents, analyze their meaning, and accurately categorize files. The system will use AI capabilities, particularly LLM, to move files to the appropriate destination folders.

1.2 Objective

- To develop a desktop application that helps users manage files efficiently with AI.
- To reduce repetitive tasks and errors caused by manual file management.
- To improve the convenience of searching and organizing files through customizable file management templates.
- To establish a foundation for future expansion toward Agentic AI features, such as file content summarization, note generation, and calendar integration.

1.3 Project Scope

The project is a desktop application that automatically manages files by using an LLM to analyze contents and move files to appropriate folders. The system must be able to:

- Support reading and analyzing PDF, Word, Excel, PowerPoint, TXT, and image files.
- Automatically categorize files and move them to designated folders based on templates.
- Provide a Preview system that allows users to review the actions before approval.
- Record operation history and support Undo/Redo for every change.
- Provide a user-friendly and secure UI, running on Windows.

CHAPTER 2 BACKGROUND, THEORIES, AND RELATED RESEARCH

2.1 Related Theories

2.1.1 Natural Language Processing (NLP)

Natural Language Processing, or NLP, is a field of artificial intelligence concerned with enabling computers to understand, analyze, and generate human-like language. NLP has various applications, such as text classification, information extraction, and summarization. In this project, NLP is applied to analyze and classify document files based on the content contained within each file.

2.1.2 Large Language Models (LLM)

Large Language Models, or LLMs, are large-scale language models trained on massive datasets to process and generate text in a way that is similar to human language. Examples of LLMs include GPT, Llama, Gemma, and others. The main strength of LLMs is their ability to understand textual context and generate complex outputs, such as document classification, text summarization, or generating new file names based on specified conditions. In this project, the LLM is used and controlled through prompt-based instructions.

2.1.3 Vision Language Models (VLM)

Vision Language Models, or VLMs, are AI systems created by combining Large Language Models with vision encoders, allowing LLMs to “see” and understand visual information. With this capability, VLMs can process and provide advanced understanding of videos, images, and text inputs provided in prompts in order to generate text-based responses.

Unlike traditional Computer Vision models, VLMs are not limited to fixed classes or specific tasks such as classification or object detection. VLMs trained on large-scale text corpora and image/video-caption pairs can be instructed using natural language and applied to many classical vision tasks, as well as new generative AI tasks such as visual summarization and answering visual question.

2.1.4 Embedding Models

Embedding Models are AI models used to convert unstructured data, such as text, images, or documents, into numerical vector representations. These vectors capture the semantic meaning of the original data, allowing computers to compare, search, and analyze information based on meaning rather than exact keyword matching. For example, two documents that use different words but discuss similar topics may have similar vector representations. This makes embedding models useful for tasks such as semantic search, document similarity comparison, file recommendation, duplicate detection, and automatic file categorization.

2.1.5 GGUF model

.gguf AI model is a compressed model file format commonly used to run LLMs locally with tools such as llama.cpp. It stores model weights, tokenizer data, and metadata in one file, making it easy to load and use offline. GGUF also supports quantization, which reduces model size and memory usage, allowing AI models to run on normal computers with limited hardware. In this case, GGUF models are useful because they allow the system to analyze, classify, summarize, and rename files locally without sending user data to cloud services. This improves privacy and supports offline AI processing.

2.1.6 Vector Database

A Vector Database is a specialized database designed to store, manage, and search data in the form of high-dimensional vectors. These vectors can capture the conceptual meaning of unstructured data, such as text, images, or audio. Instead of relying only on exact keyword matching like traditional databases, a vector database focuses on semantic similarity search. This allows the system to efficiently retrieve information that is meaningfully related to a query.

2.1.7 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation, or RAG, is a technique or process used to improve the accuracy and reliability of LLMs by retrieving additional information from trusted external sources beyond the model's training data before generating a final response. RAG is commonly used together with vector databases, which store information in numerical vector forms that AI systems can understand and retrieve efficiently.

2.1.8 Prompt Engineering

Prompt Engineering is the process of designing instruction text that is sent to an LLM in order to control the output and make it follow the desired format. For example, prompts can instruct an LLM to respond in JSON format or use few-shot examples so that the model can learn the expected pattern from sample inputs and outputs. This technique allows existing models to be used without additional fine-tuning. It is suitable for this project because the system requires accurate and clearly structured outputs, also known as structured output.

2.1.9 Hybrid Search

Hybrid Search is a search approach that combines two or more search techniques to produce more accurate and relevant results. In general, it combines keyword search, which focuses on exact word matching, with semantic search, which understands the context and meaning of text. This approach allows the system to search based on both exact words and related concepts. It can be used to help users find files more effectively by supporting various types of search queries according to user needs.

2.1.10 AI Agents

An AI Agent is a system that uses an LLM to control tasks on behalf of users. It can make decisions and perform actions automatically under defined conditions. The main strength of an AI Agent is its ability to select appropriate tools, such as calling APIs or accessing databases, and perform repetitive tasks efficiently. The main components of an AI Agent include Model, Tools, and Instructions. The Model is responsible for reasoning and decision-making, Tools are used to access information or perform actions, and Instructions define the behavior of the Agent to ensure that it operates according to the intended requirements.

2.2 Related Computer Technologies

2.2.1 Desktop applications

A desktop application is a software program installed and run directly on a personal computer or laptop, such as on Windows, macOS, or Linux, to perform specific tasks without always requiring a web browser.

2.2.2 Tauri

Tauri is a framework for developing lightweight desktop applications that can run on both Windows and macOS. It uses web technologies such as HTML, CSS, and JavaScript/TypeScript to build the user interface,

while Rust is used as the backend to connect with the file system and native APIs of the operating system. Tauri is suitable for applications that require speed, security, and a small application size.

2.2.3 React + TypeScript

React and TypeScript are used to develop the user interface of the application. React provides efficient state management. TypeScript improves code reliability by adding static type checking, which helps reduce development errors and makes the application easier to maintain.

2.2.4 Rust / Python

Rust is used to interact with the file system, manage batch processing, and handle tasks that require high performance and memory safety.
Python is used for AI-related processing, including running AI models and LLMs tasks. It is suitable for text processing, document analysis, file content extraction, and integration with machine learning libraries.

2.2.5 llama.ccp

llama.cpp is an open-source C/C++ project used to run Large Language Models locally on a computer. Its main goal is to enable LLM inference with minimal setup and good performance across different hardware. It is commonly used with GGUF models, which are model files optimized for local inference. With llama.cpp, users can run AI models without relying on cloud services, making them suitable for offline and privacy-focused applications. Hugging Face also documents GGUF usage with llama.cpp for downloading and running compatible models.

2.3 Related Research

2.3.1 Auto Organizer: A Machine Learning-Based Tool for Automatic Organization of Files

Sengar, Fatima, and Yadav proposed Auto Organizer, a machine learning-based tool for automatically organizing files on a digital device. The system was designed to solve the common problem of users having large numbers of downloaded or created files, which makes it difficult to locate specific files later. The proposed tool supports multiple file types, including PDF files, documents, images, and audio files. It applies different deep learning algorithms to classify files into predefined categories, with a total of twenty classification categories included in the system. In addition to file classification, the tool provides practical features such as directory sorting, file name recommendation, and file save path recommendation through a user-friendly graphical interface.
Relevance to the project : The basic of paper is very similar to the project “AI-powered file organizer application”, it may have some differences, but it is solid evidence that the solution to use AI models as a classifier to file organization problem is appropriate. In another sense, project “AI-powered file organizer application” is an extension and an upgrade to this paper with use of more advanced LLM models instead of simple fine-tuned machine learning.

2.3.2 Semantic File Systems

Gifford et al. introduced the Semantic File System, a file management approach that provides associative access to files by automatically extracting attributes from file contents using file-type-specific transducers. Unlike traditional hierarchical file systems, which require users to remember exact folder locations, semantic file systems allow users to locate files based on meaningful attributes such as author, title, file type, text content,

or program functions. The system also introduces virtual directories, where directory names are interpreted as queries and the results are displayed like normal folders. This design allows semantic access to remain compatible with existing file system tools and protocols. The study shows that semantic file systems can improve information retrieval and provide a more flexible storage abstraction than conventional tree-structured file systems. This research is relevant to the proposed automatic file organizer because it demonstrates the importance of content-based analysis, automatic indexing, and semantic metadata extraction for improving file discovery and organization.

Relevance to the project : To project “AI-powered file organizer application”, this paper can support the idea that file organization should not depend only on folder hierarchy or file extension. The paper provides an early theoretical and practical foundation for automatically extracting file meaning and using that information for retrieval. The project can be positioned as a modern extension of this idea by using machine learning, OCR, embeddings, or LLM-based summarization to classify and organize files automatically.

CHAPTER 3 METHODOLOGY

3.1 Project Details

This project aims to develop a desktop application that uses an LLM to automatically read, analyze, and categorize document files such as .pdf, .docx, .pptx, .xlsx and image files. The system needs to help reduce the burden of manual file management, such as moving files, renaming files, and creating new folders. Users can define their own file organization templates and review the results through a Preview screen before confirming the actual operation. The application is also designed with a History/Undo system to ensure that any changes made by the system can be reversed when needed. In addition, the application includes secondary features that apply Agentic AI concepts, such as file summarization, schedule generation, and a note-taking workspace for folders.

System Requirements

- The system must be able to read and support document files, including .pdf, .docx, .pptx, .xlsx and image files.
- The system must be able to use AI to analyze and classify files based on the content.
- The system must be able to create destination folder paths.
- The system must provide a Preview function that allows users to review the results before confirming the actual operation.
- The system must record all operation history and support Undo/Redo.
- The system must provide a user-friendly and easy-to-understand UI to improve user convenience.

3.2 System Overview

3.2.1 Dataflow Diagram (level 0)

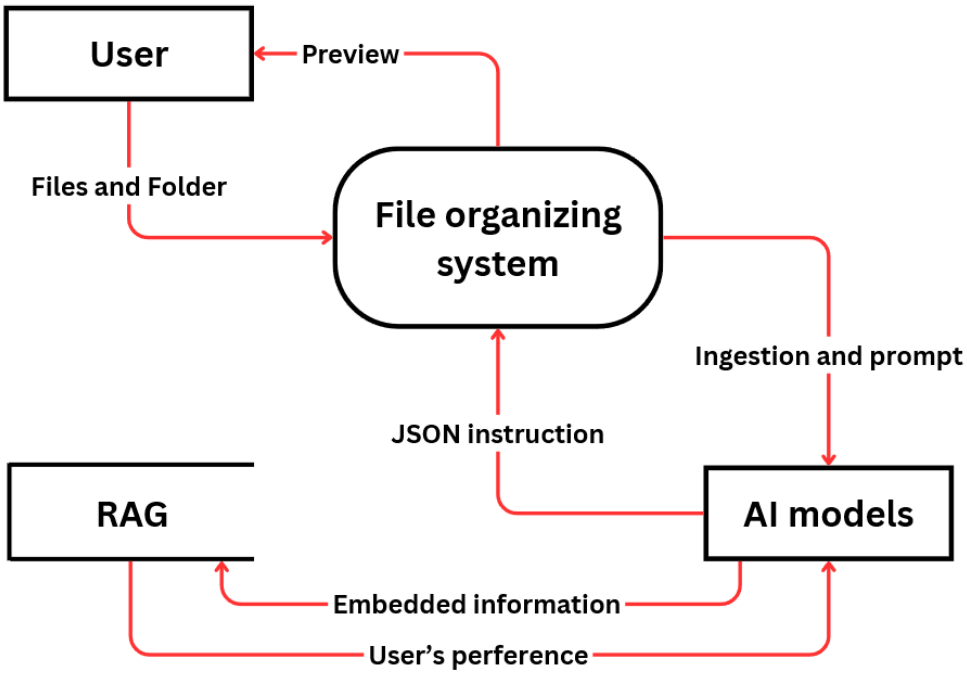

Figure 3.1: Diagram from the source document

3.2.2 Main sequence Diagram

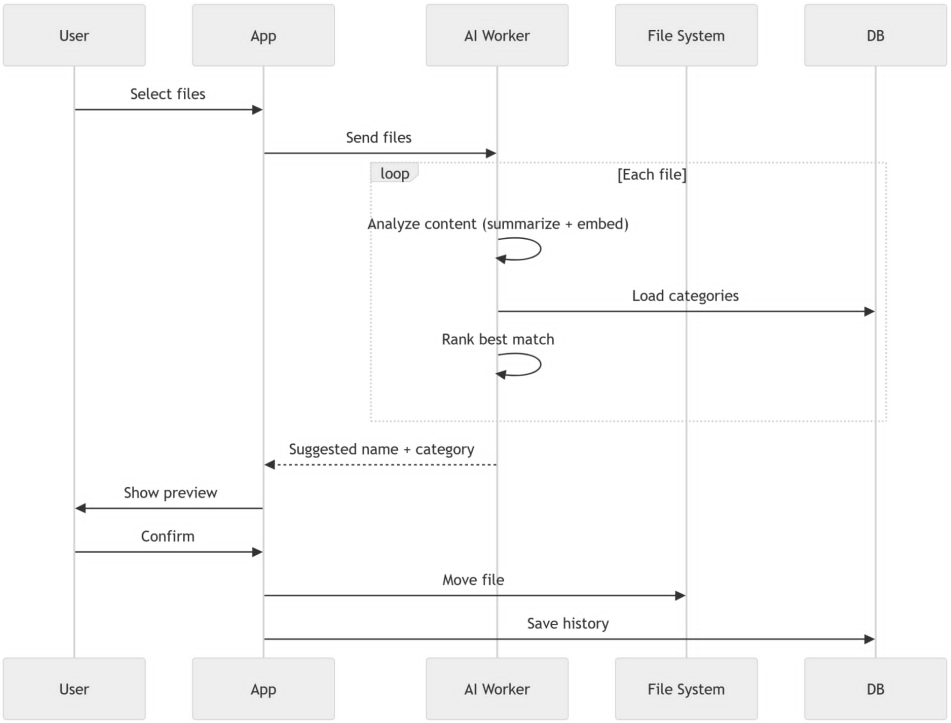

Figure 3.2: Diagram from the source document

3.2.3 System Architecture and Organization Design

This design is grounded in the Clean Architecture principle, which states that software should be organized into concentric layers where outer layers depend on inner layers, never the reverse. Each layer in the system maps to a specific responsibility:

Table 3.1: Table 2: System architecture layers and responsibilities

Layer	Component	Responsibility
UI	React/TypeScript	Render state and capture user intent
IPC	Tauri invoke()	Serialize and transmit commands across the process boundary
IPC Handler	Tauri command registration	Deserialize input, validate it, and route it to the appropriate command
Command	Rust command functions	Orchestrate one use case end-to-end
Service	Domain service classes	Encapsulate business logic
Infrastructure	File system, watcher, process manager	Execute side effects against the operating system
Repository	JSON / SQLite persistence	Read and write application state

In Tauri, the frontend and the backend (Rust) run in separate processes with no shared memory. JavaScript cannot directly access the file system, spawn processes, or interact with the operating system. All cross-boundary calls must pass through the IPC channel (invoke()), which Tauri validates against a declared capability manifest before executing. This separation naturally enforces the Separation of Concerns principle: the UI layer contains no file-system logic, and the repository layer contains no rendering logic. If the storage format changes from JSON to SQLite, only the repository layer needs to change; the command, service, and UI layers remain unaffected.

Furthermore, separating the Command layer from the Service layer allows the same business logic to be reused across multiple entry points, such as a manual user action, an automated watcher trigger, or a future scheduled task, without duplicating code.

The organization feature is the core workflow of this project. Its design addresses three fundamental challenges: files may arrive one at a time or in large batches, AI analysis is slow and non-deterministic, and file moves are irreversible so user confirmation is required before any destructive action.

The full pipeline is divided into four phases:

Phase 1: File Discovery

Files enter the organize pipeline through two mutually exclusive triggers. Manual selection starts when the user initiates a native file picker dialog through the UI. The Rust command opens the operating system's native dialog using the rfd library and returns the selected paths to the frontend. Automatic detection is used when the user has configured a watched folder. The backend registers a file system watcher using the notify crate, then applies debounce and stability checks before emitting a watched-file-added event to the frontend.

Phase 2: Frontend Queue Architecture

When files arrive, the frontend inserts each file path into a sequential analysis queue managed in the Zustand store. This queue acts as load leveling between the producer and the slow AI worker, prevents resource contention, preserves FIFO order, and allows the UI to render progress accurately.

Phase 3: AI Analysis

For each file dequeued, the frontend sends an HTTP POST request to the backend service, which runs as a separate Python process. This keeps the AI pipeline isolated from the UI, allows crashes to be handled independently, and makes future remote deployment possible without changing the frontend contract.

Phase 4: User Confirmation and File Execution

After analysis, the frontend shows ranked category suggestions, filename suggestions, and the original file path. File moves are only executed after user confirmation. The Tauri Rust backend performs the file-system operation, and the frontend records the original and new paths in the local history store so the undo flow can reverse the action later.

3.2.4 Frontend

The project is built with Tauri + React. The frontend provides a desktop UI for analyzing dropped files, suggesting categories and names, tracking organization history, and managing settings through a single cohesive interface.

Core development

- File Analysis and Intake: Drag-and-drop and file picker mechanisms for capturing files into the organization pipeline
- Interface for AI models: LLM Models configuration set up
- Historical Tracking: Comprehensive logging and searchable repository of organization activities
- Settings and Configuration: User-accessible controls for category management, folder configuration, and automation policies
- Authentication Integration: Google Calendar API access
- Background Automation: Real-time monitoring and queued file processing with concurrency control

Table 3.2: Table 1: Tools and Technology implementation in frontend

Component	**Technology**	**Rationale**
UI Framework	React	Modern concurrent rendering with automatic batching
Language	TypeScript	Compile-time type safety and IDE support
Build Tool	Vite	Rapid build times and optimized production bundles
Routing	React Router	Client-side navigation with protected route patterns
State Management	Zustand	Minimalist API with built-in persistence middleware
Component Library	shadcn/ui	Accessible, customizable Radix UI components
Styling	Tailwind CSS	Utility-first CSS with CSS variable theme support
Icons	Lucide React	Lightweight SVG icon library
Desktop Runtime	Tauri	Native desktop integration with Rust backend
Date Handling	date-fns	Functional, modular date manipulation library
Data Visualization	recharts	React-based charting library for metrics display
Notifications	Sonner	Toast notification system for user feedback
Package Manager	Bun	Fast JavaScript runtime and package manager

3.2.5 Backend

This project is a privacy-first local file organization system that assists users in categorizing and organizing files based on semantic understanding of file content. The system operates as a desktop application, processing files from absolute paths without uploading or transmitting content outside the local system boundary.

Core development

- Scanner Service: Enumerates files from specified directory paths and extracts file metadata: path, size, content hash (SHA-256), extension
- Classification: Computes cosine similarity between file embeddings and category embeddings
- RAG Service: A retrieval-augmented generation. Manages the embedding function pipeline, delegating to llama-server. Supporting semantic search
- Parser: Extracts structured text and tables from PDF, DOCX, XLSX, PPTX, HTML, and Markdown files. Supports two profiles (”fast” and ”rich”) with configurable for normal OCR or table structure extraction
- LLM Client: Async HTTP interface to out-of-process llama-server. Essential method for the access of AI. Supporter of the features such as files organization, summary
- Database: Cache on activity. Participating in workflow for the purpose of reducing workload and remembering the already processed file

Table 3.3: Table 3: Tools and technology implementation in backend

Category	**Tool**	**Purpose**
Framework	FastAPI	Web API framework
Database	SQLAlchemy + SQLModel	Object-relational mapping
Migrations	Alembic	Schema versioning
Parsing	Docling	Multi-format document parsing
RAG	RAG-Anything	Semantic indexing
RAG Backend	LightRAG	Vector storage
LLM	llama.cpp	Local LLM inference as llama-server
Vectors	NumPy	Numerical operations
Logging	logging	Structured logging
Config	dataclass + env	Settings management

AI models implementation

The system must be able to read and support document files, including .pdf, .docx, .pptx, .xlsx and image files. Some file types are easily understood by LLM because they were trained by them, as text base documents, but some like image files (png or .jpeg) cannot be comprehended by normal LLM, and that is the reason why VLM is needed.

With the integration of RAG system and vector database to support semantic search, LLM also need to embedded word from file’s content into the vector database which may interfere with its normal process and increasing more time on each session of activity. One of the solutions is to use separate embedding models to handle this job.

This project prioritizes working in local environments, that means special interface is needed. We chose llama.ccp because it requires minimal setup, suitable for offline desktop application.

In summary of this part

- All models will be of .gguf format for the use with llama.cpp
- There will be VLM + mmproj (image encoder support) and embedding model for the best practice

Workflow

The analysis pipeline inside the application applies a cache-first strategy:

- **Scan**: The file hash, size, and extension are computed.
- **Cache check**: A categories hash is computed from the current set of active categories and the parser profile. If the analysis record already exists in the database for this file with a matching hash, the cached analysis is returned immediately and no LLM call is made.
- **Parse**: If the cache misses, the file is parsed using Docling's fast profile to extract text content. The result is a structured summary suitable for LLM input.
- **Summarize**: The extracted text is sent to the llama-server chat endpoint with a summarization prompt. For image files, the vision-capable model endpoint is used instead.
- **Rename suggestion**: The summary is sent to the LLM with a rename prompt that instructs the model to produce filenames. The output is post-processed with sanitization rules to ensure filesystem compatibility.
- **Embedding and classification**: The file summary is embedded using the llama-server embedding endpoint. The resulting vector is compared against the pre-computed embedding of each active category using cosine similarity.
- **Persist**: The analysis result is written to the database and a history entry is created.

So, the basic workflow should be like this:
[Stage 1] Scan Files

- Enumerate file paths
- Compute file hashes and metadata
- Load existing File records from database
- Determine which files have changed

↓
[Stage 2] Cache Check

- If unchanged & categories match → Hit (return cached results)
- If categories changed but file unchanged → Partial (re-classify only)
- If file changed or forced → Full pipeline

↓
[Stage 3a: Full pipeline] Parse & Queue Background Ingestion

- Parsing and extracting
- Enqueue file into background RAG ingestion

↓
[Stage 3b] Generate Summary (if not cached)

- If image: send to VLM
- If text: use extracted text + LLM summary
- If neither: filename fallback

↓
[Stage 4] Generate Rename Suggestions (if summary available)

- LLM generates N candidate filenames based on summary
- Sanitize and validate suggestions

↓
[Stage 5] Classify Files

- Compute file embedding (filename + summary)
- Load active categories with embeddings
- Compute cosine similarity for each category
- Sort by score descending; retain top-K

↓
[Stage 6] Return Results

- Format response with scores, suggestions, metadata
- Log history activity

↓
[Response] (with scores, suggestions, timings)

3.3 Evaluation method

To conform with objectives of this project which is service invention project in the form of desktop application, the evaluation method must include the Performance testing and End-user opinion. So, in this case there are some of the best ways to which is:

- Satisfactory form for end-user: This method is to use form or survey to ask the end-user that we, developer of this project, distribute our application to. The number of participants should not be too low, or the result might be biased and will not express the true performance of the application
- Expert Evaluation form: This method is to use form made with technical questions to ask the expert in the field of computer engineering or software developer after we send our application to them to test. The questions must be related to the project objectives, and with an odd number of experts participating, we can also get a consensus on whether the application is successful or not.

CHAPTER 4 RESULTS AND ANALYSIS

Finished Product

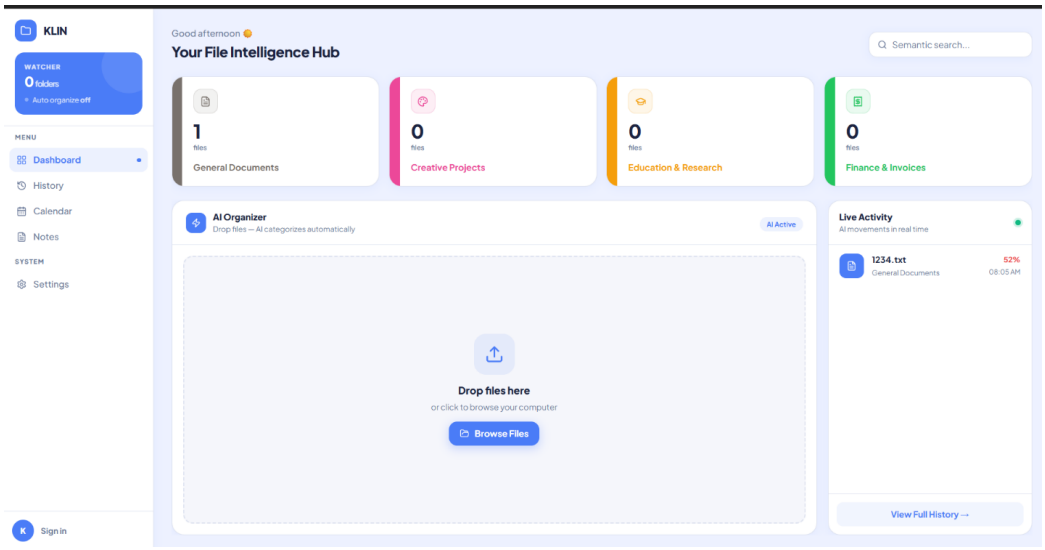

Figure 4.1: Diagram from the source document

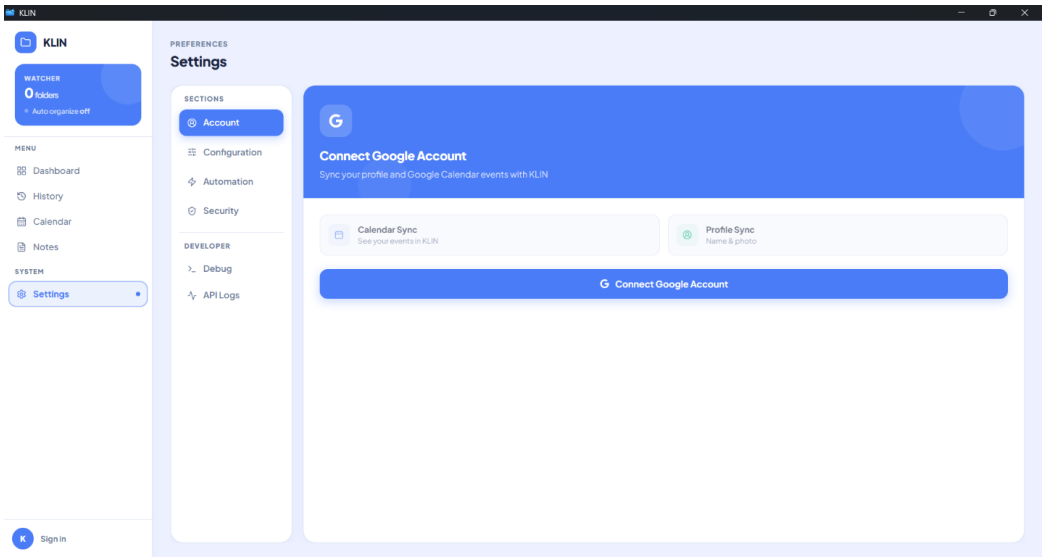

Figure 4.2: Diagram from the source document

The project “AI-powered file organizer application” was conducted according to the following steps:

- Studying project-related information
- Developing a demo application and testing the LLM model
- Creating an expert evaluation form for application performance
- Collecting data for development and improvement
- Analyzing data and summarizing the project results

Methodology

1. Expert Evaluation tool for Application Performance

4.1 The Evaluation tool used to evaluate the application was a performance evaluation form, divided into two sections:

Part 1: Performance evaluation of the “AI-powered file organizer application” by experts. The project developers defined a 5-point scoring scale with the following meanings:

- : unable to solve the problem.
- : able to solve the problem partially, but some gaps remain.
- : able to solve the problem appropriately, but further improvement is still possible
- : able to solve the problem effectively with minor improvements possible.
- : able to solve the problem completely and appropriately.

Part 2: Additional comments and suggestions on the performance evaluation of the “AI-powered file organizer application”

4.2 The data analysis in this project was conducted using computer software packages. The statistics used for analysis were as follows:

Data from the performance evaluation form of the “AI-powered file organizer application” were analyzed using the mean (X) and standard deviation (S.D.). The interpretation of the mean scores was defined as follows (Boonchom Srisa-ard, 2002: 162):

Mean score between 4.51 – 5.00 means able to solve the problem completely and appropriately.

Mean score between 3.51 – 4.50 means able to solve the problem effectively with minor improvements possible.

Mean score between 2.51 – 3.50 means able to solve the problem appropriately, but further improvement is still possible.

Mean score between 1.51 – 2.50 means able to solve the problem partially, but some gaps remain.

Mean score between 1.00 – 1.50 means unable to solve the problem.

Data Analysis

The project developers analyzed data from the expert performance evaluation form for the “AI-powered file organizer application”. The evaluation was conducted by 3 experts. The analysis results were presented in tables together with descriptive explanations and divided into two parts as follows:

Part 1: Analysis results of the expert performance evaluation of the “AI-powered file organizer application”. The analysis used the mean and standard deviation, both overall and by individual evaluation items. To make the data analysis easier to understand, the developer defined the statistical symbols and abbreviations used in the study as follows:

n refers to the sample size.X refers to the mean score of satisfaction.(S.D.) refers to the standard deviation.

Part 2: Additional comments and suggestions on the performance evaluation of the “AI-powered file organizer application”.

Data Analysis Results

Part 1: Analysis results of the expert performance evaluation of the “AI-powered file organizer application” using the mean and standard deviation, both overall and by individual evaluation items.

Table 4.1: Table 2: Mean, standard deviation, and analysis of expert opinions regarding the performance of the “AI-powered file organizer application”

Evaluation Items	Result		
	X	SD	Interpretation
1. The system can accurately read and analyze data from PDF, Word, Excel, PowerPoint, TXT, and image files.	4.33	0.57	(4)
2. The system can accurately and appropriately categorize files based on their content.	3.66	0.47	(4)
3. The system can accurately and appropriately move files to destination folders according to defined templates.	5	0	(5)
4. The Preview system effectively helps reduce errors before moving or modifying files.	5	0	(5)
5. The history recording and Undo/Redo system is appropriate for practical use.	4	0	(4)
6. The UI design is easy to learn and use.	3.51	0.47	(4)
7. AI-related features, such as content summarization, note generation, and search are appropriate for practical use.	4	0	(4)
8. The system provides appropriate measures to prevent risks to user data.	5	0	(5)
9. The overall system architecture is appropriate for development as a desktop application.	4	0	(4)
10. The system has the potential to support real-world use on Windows.	4.66	0.47	(5)
Total	4.31	0.19	(4)

From Table 2, it was found that the experts’ opinions regarding the performance of the “AI-powered file organizer application” were overall at the level of able to solve the problem effectively with minor improvements possible, with a mean score of 4.31. The standard deviation was 0.19, indicating a low level of dispersion from the mean. This means that the experts’ opinions regarding the application’s performance were similar and consistent. And when considering each evaluation item individually, four items were rated at the level of able to solve the problem completely and appropriately. These items were: The system can accurately and appropriately move files to destination folders according to defined templates, The Preview system effectively helps reduce errors before moving or modifying files, The system provides appropriate measures to prevent risks to user data, The system has the potential to support real-world use on Windows. The remaining evaluation items were rated at the level of able to solve the problem effectively with minor improvements possible.

Part 2: Additional Comments and Suggestions on the Performance Evaluation of the “AI-powered file organizer application”

Expert “1” evaluated that this application is of very good quality. This reflects a clear understanding of applying a Local LLM to solve real-world problems. The application has several strengths, including:

Strengths

- Use of Offline AI (llama.ccp) – This is an important strength, especially for organizations that require privacy and do not want to rely on cloud services.
- Preview & Undo System – This is very well designed and helps reduce the risk of incorrect file organization. It also provides good user experience.
- Full Thai Language Support – The system supports Thai documents, showing attention to the local context.
- Hybrid Approach (Keyword + AI) – Using keyword matching first and then sending the task to AI is a good concept. It helps save time and computational resources.

Areas for Improvement

- UI/UX – Although the application is usable, it is recommended to further study Material Design or Fluent Design, and Dark Mode may also be considered.
- Error Handling – Error handling should be clearer. For example, if AI models are not running or some process fails, the system should display easy-to-understand messages.
- Documentation – A complete user manual should be provided, including a troubleshooting guide.

Additional Suggestions

- Consider adding an adjustable Confidence Score Threshold for users, such as asking the user for confirmation if the AI confidence score is lower than 70%.
- Add a Batch Rename feature to rename files according to a predefined format.
- Develop an Analytics Dashboard to display file organization statistics, such as the number of files in each category and total file size.

Overall Comment

This application has academic value and can be applied in real-world use. It is recommended that the project be further developed as an Open-Source Project. The developers may also consider publishing a paper on the Hybrid Keyword-AI Approach for Thai-language documents.
Expert “2” evaluated that this “AI-powered file organizer application” is highly outstanding and useful, especially in educational and real-world working contexts.

Strengths

- Practical Usability – This is not only a sample project, but an application that can be immediately used to solve real problems in offices or government agencies, where large numbers of documents often need to be organized.
- Thai document parsing – This is a major strength because most government documents are in Thai and often include scanned documents that require OCR. The ability to support Thai language well makes the application practical for real use.
- 100% Offline Operation – This is very important for organizations that handle sensitive information, as they do not need to worry about data security risks from cloud services.

- Preview and Undo Features – Students often develop projects without considering these features, but this project clearly takes real users into account.

Application in Teaching

This project is a very good example for students to learn from in the following areas:

- Machine Learning Integration – It demonstrates integration with the AI models.
- File Handling – It involves managing multiple file formats.
- Real-World Problem Solving – It solves a real problem rather than simply following a basic example.

Areas for Improvement

- Create Tutorial Videos – Tutorial videos should be created to help general users understand how to use the application more quickly, especially users who are not familiar with technology.
- Improve the UI for Easier Use – Although the application is already usable, it should include a wizard or guided setup for first-time users.
- Add Use Case Examples – Preset templates should be provided, such as “organizing human resources documents” or “organizing financial documents.”

Additional Suggestions

- Consider adding multi-language support, starting with Thai and English, which has already been implemented, and possibly adding more languages in the future.
- Add Export/Import Settings so that users can share templates with their colleagues.
- Develop a Lite Version for low-specification computers, which may use only keyword matching without AI.

Overall Comment

This “AI-powered file organizer application” is highly suitable for presentation at an academic conference. It should also be considered for release as an Open-Source Project so that the community can benefit from it. This would also help build recognition for the institution and the developers.
Expert “3” evaluated this application from the perspective of real-world organizational use and data security.

Strengths

- Offline-first Architecture – Data does not leave the user’s device, making it suitable for organizations that handle sensitive information.
- Local AI Processing – This helps reduce the risk of data breaches.
- Password-protected File Support – The application takes data security into consideration.
- System Architecture – The code structure is well organized, and the installer is ready for use.

Areas for Improvement for Enterprise Deployment

- Security and Access Control – User authentication and role-based access control should be added for organizational use.
- Audit Trail – Logs should include more details, such as User ID, IP address, and file hash for integrity checking.
- Scalability – A database such as SQLite should be considered instead of JSON files when handling a large number of files.
- Network Deployment – A client-server architecture should be developed to allow multiple users to work together more conveniently.
- Backup and Recovery – Auto-backup mechanisms for configuration and export/import settings should be provided.

Additional Suggestions

- Consider Active Directory integration for organizations using a Windows Domain.
- Add API endpoints so that other systems can access and use the application.
- Develop a Health Check Dashboard for IT administrators to monitor system status.
- Add unit tests for critical functions.

Summary

The application has a very strong foundation and is ready for personal use or small teams. However, if it is intended for use in large organizations, additional enterprise features should be developed according to the suggestions above. The use of Offline AI is an important strength in the on-premises solution market.

CHAPTER 5 CONCLUSION AND RECOMMENDATIONS

The project titled “AI-powered file organizer application” can be summarized, discussed, and presented with recommendations as follows:

Project Summary

Project Objectives

- To develop a desktop application that helps users manage files efficiently with AI.
- To reduce repetitive tasks and errors caused by manual file management.
- To improve the convenience of searching and organizing files through customizable file management templates.
- To establish a foundation for future expansion toward Agentic AI features, such as file content summarization, note generation, and calendar integration.

Methodology

1. Expert Evaluation tool for Application Performance

5.1 The Evaluation tool used to evaluate the application was a performance evaluation form, divided into two sections:

Part 1: Performance evaluation of the “AI-powered file organizer application” by experts. The project developers defined a 5-point scoring scale with the following meanings:

1. : unable to solve the problem.
2. : means able to solve the problem partially, but some gaps remain.
3. : means able to solve the problem appropriately, but further improvement is still possible
4. : means able to solve the problem effectively with minor improvements possible.
5. : means able to solve the problem completely and appropriately.

Part 2: Additional comments and suggestions on the performance evaluation of the “AI-powered file organizer application”

5.2 The data analysis in this project was conducted using computer software packages. The statistics used for analysis were as follows:

Data from the performance evaluation form of the “AI-powered file organizer application” were analyzed using the mean (X) and standard deviation (S.D.). The interpretation of the mean scores was defined as follows (Boonchom Srisa-ard, 2002: 162):

Mean score between 4.51 – 5.00 means able to solve the problem completely and appropriately.

Mean score between 3.51 – 4.50 means able to solve the problem effectively with minor improvements possible.

Mean score between 2.51 – 3.50 means able to solve the problem appropriately, but further improvement is still possible.
Mean score between 1.51 – 2.50 means able to solve the problem partially, but some gaps remain.
Mean score between 1.00 – 1.50 means unable to solve the problem. The result from the Expert Evaluation from stated that the experts' opinions regarding the performance of the "AI-powered file organizer application" were overall at the level of able to solve the problem effectively with minor improvements possible, with a mean score of 4.31. The standard deviation was 0.19, indicating a low level of dispersion from the mean. This means that the experts' opinions regarding the application's performance were similar and consistent. And when considering each evaluation item individually, four items were rated at the level of able to solve the problem completely and appropriately. These items were: The system can accurately and appropriately move files to destination folders according to defined templates, The Preview system effectively helps reduce errors before moving or modifying files, The system provides appropriate measures to prevent risks to user data, The system has the potential to support real-world use on Windows. The remaining evaluation items were rated at the level of able to solve the problem effectively with minor improvements possible.

Summary of Expert Opinions on the Performance Evaluation of the "AI-powered file organizer application"

- This application has academic value and can be applied in real-world use. It is recommended that the project be further developed as an Open-Source Project. The developers may also consider publishing a paper on the Hybrid Keyword-AI Approach for Thai-language documents.
- This "AI-powered file organizer application" is highly suitable for presentation at an academic conference. It should also be considered for release as an Open-Source Project so that the community can benefit from it. This would also help build recognition for the institution and the developers.
- The application has a very strong foundation and is ready for personal use or small teams. However, if it is intended for use in large organizations, additional enterprise features should be developed according to the suggestions above. The use of Offline AI is an important strength in the on-premises solution market.

Project Recommendations

1. Consider adding an adjustable Confidence Score Threshold, such as asking the user for confirmation if the AI confidence score is lower than 70%.
2. Add a Batch Rename feature to rename files according to a predefined format.
3. Develop an Analytics Dashboard to display file organization statistics, such as the number of files in each category and the total file size.
4. Consider adding Multi-language Support, starting with Thai and English, which has already been implemented, and possibly adding more languages in the future.
5. Add Export/Import Settings so that users can share templates with their colleagues.
6. Develop a Lite Version for low-specification computers, which may use only keyword matching without AI.
7. Consider Active Directory Integration for organizations using a Windows Domain.

8. Add API Endpoints so that other systems can access and use the application.
9. Develop a Health Check Dashboard for IT administrators to monitor system status.
10. Add Unit Tests for critical functions.

REFERENCES

1. AWS, 2024, “What is Natural Language Processing (NLP)? - AWS,” Available at https://aws.amazon.com/th/what-is/nlp/, [Online; accessed 4-May-2026].

2. AWS, 2024, “What is LLM? - Large Language Model Explanation - AWS,” Available at https://aws.amazon.com/th/what-is/large-language-model/, [Online; accessed 4-May-2026].

3. NVIDIA, 2024, “What are Vision-Language Models? | NVIDIA Glossary,” Available at https://www.nvidia.com/en-us/glossary/vision-language-models/, [Online; accessed 4-May-2026].

4. AWS, 2024, “What is Retrieval-Augmented Generation (RAG)? - AWS,” Available at https://aws.amazon.com/th/what-is/retrieval-augmented-generation/, [Online; accessed 4-May-2026].

5. Ollama, 2026, “Ollama,” Available at https://ollama.com/, [Online; accessed 4-May-2026].

6. Hugging Face, 2026, “Hugging Face – The AI community building the future,” Available at https://huggingface.co/, [Online; accessed 4-May-2026].

7. Z. Guo, X. Ren, L. Xu, J. Zhang, and C. Huang, 2025, “RAG-Anything: All-in-One RAG Framework,” arXiv:2510.12323 [cs.AI], Oct. 2025. Available at https://arxiv.org/abs/2510.12323, [Online; accessed 4-May-2026].

8. HKUDS, 2026, “GitHub - HKUDS/RAG-Anything: ’RAG-Anything: All-in-One RAG Framework’,” Available at https://github.com/HKUDS/RAG-Anything, [Online; accessed 4-May-2026].

9. HKUDS, 2026, “GitHub - HKUDS/LightRAG: [EMNLP2025] ’LightRAG: Simple and Fast Retrieval-Augmented Generation’,” Available at https://github.com/HKUDS/LightRAG, [Online; accessed 4-May-2026].

10. Python Software Foundation, 2026, “Our Documentation | Python.org,” Available at https://www.python.org/doc/, [Online; accessed 4-May-2026].

11. The Rust Project Developers, 2018, “Documentation - The Rust Programming Language - MIT,” Available at https://web.mit.edu/rust-lang_v1.25/arch/amd64_ubuntu1404/share/doc/rust/html/book/first-edition/documentation.html, [Online; accessed 4-May-2026].

12. Tauri, 2026, “Create a Project - Tauri,” Available at https://v2.tauri.app/start/, [Online; accessed 4-May-2026].

13. D.K. Gifford, P. Jouvelot, M.A. Sheldon, and J.W. O’Toole Jr., 1991, “Semantic File Systems,” in Proc. 13th ACM Symposium on Operating Systems Principles, pp. 16–25, Oct. 1991. Available at https://pages.cs.wisc.edu/~remzi/Classes/838/Fall2001/Papers/sfs-sosp91.pdf, [Online; accessed 4-May-2026].

14. N. Sengar, R.C. Joshi, and M.K. Dutta, 2023, “Auto Organizer: A Machine Learning-Based Tool for Automatic Organization of Files,” Lecture Notes in Electrical Engineering, Jun. 2023. Available at https://www.researchgate.net/publication/371930502_Auto_Organizer_A_Machine_Learning-Based_Tool_for_Automatic_Organization_of_Files, [Online; accessed 4-May-2026].

15. ggml-org, 2026, “ggml-org/llama.cpp: LLM inference in C/C++ - GitHub,” Available at https://github.com/ggml-org/llama.cpp, [Online; accessed 4-May-2026].

APPENDIX A

EXPERTS' FORM

APPENDIX B

INSTALLATION GUIDE